

GlyphBreaker

Structured Adversarial Testing for Large Language Models

Multi-Provider Red-Teaming and Defense Analysis Workbench

Technical Documentation

Ritvik Indupuri

Abstract

GlyphBreaker is a client-side workbench for controlled adversarial testing of large-language-model behavior. It combines a scenario library, editable target instructions, model and sampling controls, simulated agent-tool definitions, a prompt debugger, streaming conversations and a structured defense-analysis view in one React application.

A tester can direct the same conversation workflow to Google Gemini, OpenAI, Anthropic Claude or a local Ollama server. Gemini also supports three auxiliary tasks: generating realistic simulated tool definitions, proposing the next adversarial prompt and producing an AISecOps-style analysis of the captured conversation. Session history, custom scenarios, provider settings and completed-response cache entries remain in the browser.

Document scope

- Application structure, model-provider boundaries and browser-local state.
- Pre-send preparation, provider fan-out, response fan-in and analysis flow.
- Scenario coverage, prompt construction and simulated agent-tool behavior.
- Provider adapters, streaming formats, caching and structured rendering.
- Implemented security boundaries, operation and source cross-reference.
- Prioritized next steps for reliability, testability and safer deployment.

Contents

Abstract	2
1 System overview	3
2 System architecture	4
3 Adversarial request lifecycle	6
4 Attack library and test context	7
5 Prompt construction and response rendering	8
6 Provider routing and streaming	8
7 Gemini-assisted red-team and defense roles	9
8 Session state, history and credentials	9
9 Security and operational boundaries	10
10 Build and operation	10
11 Implementation cross-reference	11
12 Next steps	11

1 System overview

GlyphBreaker organizes an LLM security exercise around a mutable session. The session contains conversation messages, a target system prompt, provider configuration and zero or more simulated tool definitions. Selecting an attack template can populate the user payload, target instructions and suggested tools; the tester can then inspect or modify each element before sending it to the selected model.

The tool definitions are serialized into the model's system prompt. GlyphBreaker asks the model to describe a tool call, but the application does not execute the named tool or its arguments.

1.1 Implemented capabilities

Capability	Implementation
Scenario-driven testing	Built-in OWASP-aligned and agentic attack templates plus browser-saved custom templates.
Target configuration	Editable system prompt, provider, model, temperature, top-p and top-k controls.
Multi-provider chat	Streaming Gemini, OpenAI, Anthropic and Ollama adapters behind one service interface.
Adaptive red teaming	Gemini produces the next candidate attack prompt from the goal and prior conversation.
Defense analysis	Gemini converts the conversation into structured threat, impact and hardening sections.
Local continuity	Browser LocalStorage retains history, templates, settings, keys and optional response cache.

1.2 Technology profile

Layer	Technology
Application	React 19, TypeScript and Vite
Model access	Google Generative AI SDK; browser fetch adapters for OpenAI, Anthropic and Ollama
State	React hooks and browser LocalStorage
Presentation	Responsive component layout and structured text, table and chart rendering
Quality commands	TypeScript no-emit lint, Vite build and preview scripts

2 System architecture

Figure 1 separates pre-send controls, client orchestration, provider endpoints and browser-local persistence. Agent icons identify all LLM-backed nodes: Gemini, OpenAI, Claude and Ollama. Software components, controls and storage remain icon-free.

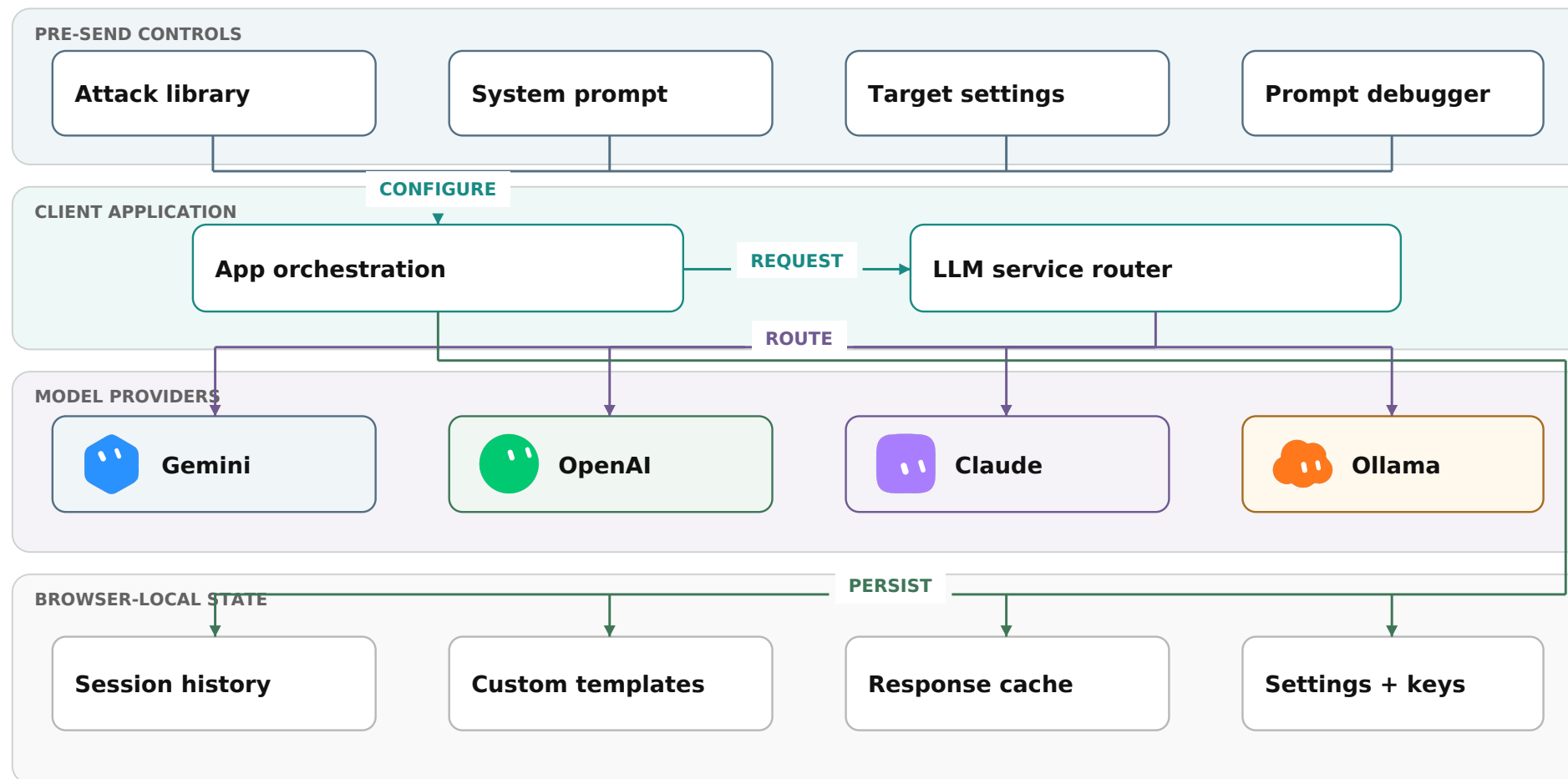


Figure 1: GlyphBreaker client architecture and external model-provider boundary.

2.1 Pre-send control surface

The control panel exposes the attack library, target system prompt, model configuration and simulated tools. The prompt debugger provides a separate inspection surface for the active system prompt, user payload, conversation and tools before submission. Separating these controls makes the effective test context visible and editable rather than hiding it inside a single request builder.

2.2 Application and service boundary

App.tsx owns session state and coordinates the three primary work areas: controls, chat and defense analysis. services/llmService.ts converts that state into provider-specific requests, performs cache lookup, streams response fragments and stores completed outputs. No application backend is present in the repository; browser code communicates directly with configured provider endpoints.

2.3 Model-provider boundary

Gemini uses the Google Generative AI SDK. The auxiliary suggestion, tool-generation and defense-analysis calls use the source-configured gemini-3-flash-preview identifier; target chat uses the selected model. OpenAI and Anthropic use their respective browser HTTP APIs. Ollama uses the local /api/chat interface, with http://localhost:11434 as the default base URL. The selected target provider affects chat generation; the auxiliary red-team and analysis functions remain Gemini-backed.

2.4 Browser-local state

GlyphBreaker persists API configuration, cache preference, session history and custom attack templates under glyph-prefixed LocalStorage keys. Cached model responses are stored separately. This design keeps deployment simple, but persistence and credential confidentiality inherit the security of the browser profile and the page executing the application.

3 Adversarial request lifecycle

Figure 2 shows the live-request path with four pre-send components, provider selection and streamed-response collection. A cache hit bypasses provider execution. Agent icons appear only on LLM-backed providers and Gemini-backed analysis roles. Every connector label is placed in an opaque gap rather than printed over the line.

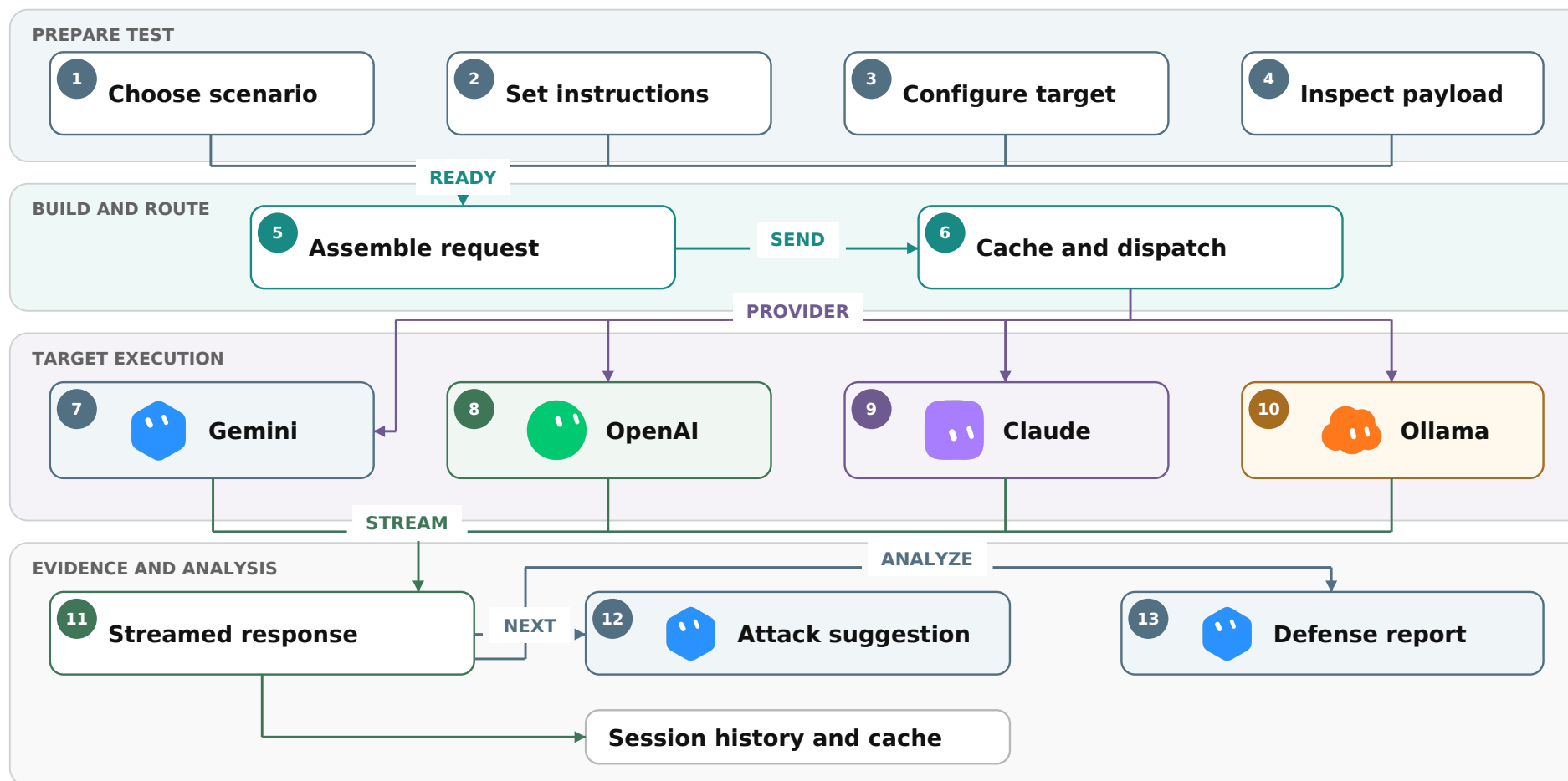


Figure 2: Request preparation, target routing and optional Gemini analysis branches.

3.1 Prepare and inspect

The tester chooses a built-in or custom security scenario, sets the target's system instructions, chooses a provider and model, adjusts supported sampling parameters and reviews the resulting payload. A selected template may also contribute an attack goal, suggested system prompts and simulated tools. The debugger can flatten selected context into a revised prompt before the request is sent.

3.2 Route and stream

The application appends the new user message to session history and asks the LLM service to stream a response. The service derives a JSON-serialized cache key from the provider, model, prompts, history and sampling configuration; it does not hash that key. A hit is replayed to the interface in chunks; otherwise the selected adapter initiates a live request and yields provider-specific stream fragments through a common asynchronous generator.

3.3 Continue or analyze

The streamed assistant response becomes part of the active conversation. In adversarial mode, Gemini can evaluate that conversation against the selected goal and write a proposed next user prompt into the composer; the tester remains responsible for sending it. Independently, the defense panel can submit the conversation to Gemini for a structured security assessment and render the returned sections as analyst-readable evidence.

4 Attack library and test context

4.1 Built-in coverage

The template catalog covers direct and indirect prompt injection, insecure output handling, training-data poisoning, model denial of service, supply-chain risk, sensitive-information disclosure, insecure plugin design, excessive agency, overreliance, model theft and agent-focused scenarios such as cyber-espionage, tool-choice manipulation and cross-plugin escalation. Each template can provide a description, user payload, attack goal, suggested system prompts and suggested tool definitions.

4.2 Custom scenarios

Users can create, update and delete custom attack templates. New entries receive UUIDs and are persisted in `glyph_customTemplates`. Selecting a template updates the composer and, when available, the first suggested system prompt and suggested tools. This allows repeatable exercises without changing source code.

4.3 Simulated agent tools

Tool definitions contain an identifier, name, description and JSON-encoded parameter schema. Before a target request, GlyphBreaker serializes these definitions into the system prompt and instructs the model to return a tool name and arguments when appropriate. Gemini can generate three to five scenario-aligned tool definitions, but generated definitions are still descriptive test inputs—not executable integrations.

Context element	Purpose	Authority
System prompt	Defines target behavior and output format	Tester editable

Context element	Purpose	Authority
User payload	Carries the adversarial test input	Tester sends
Attack goal	Guides Gemini's next-step suggestion	Template or tester
Tool definitions	Simulate available agent capabilities	Prompt context only
Conversation	Provides evidence and adaptive context	Session state

5 Prompt construction and response rendering

5.1 Target request assembly

The target adapter receives the selected provider, ordered messages, provider credentials, system prompt, model configuration, cache setting and simulated tools. Tool context is appended to the system instruction. Gemini receives a native system instruction and mapped user/model history; OpenAI, Anthropic and Ollama receive equivalent provider-specific message structures.

5.2 Structured output convention

New sessions add an instruction that general prose use SECTION: and BULLET: markers, while table and chart responses use exact JSON envelopes. ChatPanel recognizes these formats and renders structured sections, tables and bar charts. Responses that do not match a structured envelope remain visible as ordinary text.

5.3 Prompt debugger

The debugger displays the active instructions, payload, prior messages and tool definitions in a dedicated modal. Its keyword highlighting supports review of common injection language, and the edited result can be applied back to the session system prompt. This is an inspection and editing aid; it is not a policy enforcement engine.

6 Provider routing and streaming

Provider	Transport	Implemented stream handling
Gemini	Google Generative AI SDK	Iterates SDK content chunks from the configured model.
OpenAI	POST /v1/chat/completions	Parses server-sent data records from a streaming response.
Anthropic Claude	POST /v1/messages	Parses Anthropic streaming events through direct browser access.
Ollama	POST /api/chat	Parses newline-delimited JSON from the configured local base URL.

6.1 Connectivity checks

The service includes status checks for Ollama, OpenAI and Anthropic model endpoints. Ollama queries /api/tags, OpenAI queries /v1/models and Anthropic queries /v1/models with its direct-browser header. Provider errors are normalized into user-visible Error messages in the conversation while the loading state is cleared.

6.2 Cache behavior

When caching is enabled, the service uses a JSON serialization of provider, model, system prompt, message history and sampling values. A completed live response is stored under that key; a later hit is replayed in fixed-size fragments so the UI follows the same incremental update path.

Disabling cache forces a live provider request.

The current cache identity does not include the simulated tool list. Tests that reuse identical prompts with different tools should keep caching disabled until tool definitions are included in the key.

7 Gemini-assisted red-team and defense roles

7.1 Adaptive attack suggestion

streamAdversarialSuggestion supplies Gemini with the attack goal, target system prompt, simulated tools and formatted conversation history. The model is instructed to return only the next user prompt and to adapt when the target resists. The suggestion streams into the chat input field; it does not bypass the tester or automatically submit itself.

7.2 Defense analysis

streamAnalysis sends the captured conversation to Gemini with an AISECops instruction. The required response covers executive summary, threat-vector analysis, impact assessment and defense synthesis using SECTION: and BULLET: markers. DefenseAnalysisPanel parses those markers and displays the report as structured sections.

7.3 Generated tool context

generateAgentTools asks Gemini for a JSON array of realistic, high-agency tool definitions aligned to OWASP LLM07/LLM08 and MITRE ATLAS themes. The returned JSON is parsed into the session's tool list and can be edited before use. The function expects three to five tools and rejects responses that cannot be parsed as JSON.

8 Session state, history and credentials

8.1 Active and saved sessions

An active session contains its UUID, display name, ordered messages, target system prompt, LLM configuration and simulated tools. Saving inserts or updates the session in history. Clearing a non-empty session moves it into history and creates a fresh session; restoring removes the selected entry from history and makes it active.

8.2 LocalStorage inventory

Key or family	Stored content
glyph_apiKeys	OpenAI and Anthropic keys plus Ollama configuration
glyph_cacheEnabled	Boolean response-cache preference
glyph_sessionsHistory	Saved sessions and conversation evidence
glyph_customTemplates	User-authored attack scenarios
Response-cache entries	Completed target responses indexed by request identity

8.3 Credential path

Vite substitutes the Gemini API key into the client bundle at build time. The Gemini functions read that substituted API_KEY value. OpenAI and Anthropic credentials are supplied through the interface and persisted with the rest of provider settings. Because the browser sends requests directly, deployed use should assume that any script running in the same origin can access browser-managed configuration.

9 Security and operational boundaries

Boundary	Implemented behavior
Execution	Client-side application; no repository backend or server-side credential vault.
Tools	Definitions influence model context; no tool dispatcher executes returned calls.
Input handling	Chat rendering strips HTML-like tags before display; prompts remain adversarial test material.
Analysis	Gemini outputs recommendations; the tester interprets and decides how to proceed.
Persistence	History, templates, settings, credentials and cache inherit browser-profile security.
Local model	Ollama defaults to localhost and requires compatible browser/CORS configuration.

Use GlyphBreaker only against models, prompts, accounts and endpoints you are authorized to test. Red-team outputs can contain unsafe instructions or sensitive conversation data and should be handled as security evidence.

10 Build and operation

10.1 Local workflow

```
npm install

# Configure API_KEY for Gemini-backed functions
npm run dev

# Static validation and production bundle
npm run lint
npm run build
npm run preview
```

10.2 Recommended test sequence

- Select a scenario and read its goal before applying the supplied prompt or tools.
- Configure the target provider, model and sampling values, then inspect the effective context.
- Send the payload and preserve the complete target response in the active session.
- Use adaptive suggestion only when another controlled turn is required; review it before sending.
- Run defense analysis after sufficient evidence exists and save the session for comparison.
- Disable caching when changing simulated tools or when a fresh target response is required.

10.3 Provider prerequisites

A target test requires credentials or a reachable local model for the selected provider. Gemini-backed attack suggestions, defense reports and tool generation require API_KEY even when the target conversation uses OpenAI, Claude or Ollama. Direct browser requests must also be allowed by the provider and the deployment's browser security policy.

11 Implementation cross-reference

Source reference: [GlyphBreaker](#), revision 203667cdb0e8. The review covers source and configuration; it does not certify a live deployment or every runtime outcome.

This map connects the documented behavior to the repository source.

Capability	Primary implementation
Session orchestration and persistence	App.tsx
Provider adapters, cache and Gemini roles	services/llmService.ts
Provider and session data contracts	types.ts
Attack templates and output protocol	constants.ts
Target configuration and tool controls	components/ControlPanel.tsx
Streaming chat and structured rendering	components/ChatPanel.tsx
Threat-report parsing and display	components/DefenseAnalysisPanel.tsx
Prompt inspection and editing	components/PromptDebuggerModal.tsx
Saved-session management	components/SessionHistoryModal.tsx
Custom attack templates	components/CustomTemplateManagerModal.tsx
Ollama browser guidance	components/OllamaCorsModal.tsx
Build and validation commands	package.json; vite.config.ts; tsconfig.json

12 Next steps

Priority	Improvement	Value
1	Move production credentials behind a server-side relay	Avoid persistent browser exposure and centralize provider policy.
2	Include simulated tools in the cache identity	Prevent cached responses from crossing different tool contexts.
3	Add provider-adapter contract tests	Verify message mapping, stream parsing, cancellation and error handling.
4	Add scenario regression fixtures	Lock template metadata, structured rendering and expected workflow behavior.
5	Export sessions as portable evidence	Support review, comparison and retention outside one browser profile.
6	Separate target and analysis credentials	Make provider purpose and access scope explicit in deployment.